

GENG5511 MPE Engineering Research Project Part 1
Project Proposal

**Development of JavaScript based web system for displaying
operational ocean - atmosphere model results.**

Jared Healy

23398223

School of Engineering, University of Western Australia

Supervisor: Ivica Janekovic

School of Engineering, University of Western Australia

Word count: 7958

School of Engineering
University of Western Australia

Submitted: 20 May 2026

© March 2026

STUDENT DECLARATION

My contribution to the project

Provide details here of your contribution to the project and the contribution of others. See Section 2.3 of this guide for more information.

Use of AI tools

I have used AI tools in the preparation of my report: Yes/No

Enter details of how AI tools were used:

N/A

In accordance with University Policy, I certify that:

The above information is correct, and the attached work submitted for assessment is my own work and that all material drawn from other sources has been fully acknowledged and referenced.

Student signature Jared Healy Date 20/05/2026

Supervisor confirmation

To the best of my knowledge, the student's contribution outlined above is correct.

Supervisor signature _____ Date _____

Project Summary

The aim of this project is to display data generated from weather forecasts on a website in a way which is easy to access and easy to understand. The system will use TypeScript to display NetCDF and GRIB files in a visual format, in a style that is similar to *windy.com*. The project will provide an easy way for academics and laypersons to use to understand the meteorological data which weather systems record

The website will use tools such as *leaflet.js* to show render the data provided onto the map, with *react-leaflet* being used to allow for the *leaflet* to be used in the tech stack. A *Next.js* backend will be used to host the site, and a front-end user interface (UI) framework such as *shadcn/ui* will be used for making the rest of the website. These tools are open source, meaning that the project will be built in an entirely free and open-source platform. On top of this, using *Next.js* means that hosting through Vercel is easy and – for personal, non-commercial use – is free (Vercel, 2025), and only costs \$20 a month for commercial purposes.

The website will support the viewing of wind speed and direction, temperatures, rainfall, swell as well as other meteorological and oceanographic data metrics. It will display this data in toggleable layers, allowing you to view the data that is most relevant, as well as supporting custom data sources, allowing you to quickly verify that your data sources are functional.

Acknowledgements

Firstly, I would like to thank Ivica Janekovic, my supervisor, for putting up with me and helping me throughout the development process. I know that I wasn't always the easiest student to work with, but you stuck with me and I appreciate that you were willing to help me after I took so long to help myself. From the bottom of my heart, thank you.

I would also like to thank my parents, for listening to my insane ramblings about curvilinear grids and SVG renderers. I'm sure that I've driven them insane throughout this project and I'm equally sure that I will continue to drive them insane in the future.

Finally, I'd like to thank my best friend Digby, for letting me explain my issues to him, only for me to figure it out halfway through explaining it, and for supporting me throughout the project, and my girlfriend Kaliha, who has had to sit on my bed for hours whilst I tapped away at both the project and this report

Contents

STUDENT DECLARATION.....	ii
Project Summary.....	iii
Contents.....	v
1. Introduction.....	1
1.1. Literature Review	1
1.2. Project Objectives.....	3
2. Project Process	3
2.1. Constraints	3
2.2. Design Criteria	3
2.3. Software Stack.....	4
2.4. Design Evaluation	5
2.5. Health & Safety Considerations	5
3. Results and Discussion	7
3.1. Final Design	7
3.2. Performance and Analysis	13
4. Conclusions and Future Work	14
5. References.....	16
6. Appendices.....	18
Appendix A: Raw Data for Performance Measurements	18

List of Figures

Figure 1 An example of a minimised chunk used by windy.com.....	2
Figure 2: Ventusky's visual representation of weather from 04/10/2025.....	2
Figure 3: The final renderer, representing the windspeed and the surface temperature	7
Figure 4: The particle renderer showing wind data on April 1 st of 2026.....	8
Figure 5: Wind barbs for 0, 5, 10, 15, 20, 50, 55, 60 and 100 knots	8
Figure 6: Temperature data being displayed for the 20 th of January (Hottest day of 2026 at time of writing).....	9
Figure 7: The ocean temperature and current arrows being drawn behind Rottneest	10
Figure 8: A closeup of the point grapher, comparing the temperature at different points	11

List of Tables

Table 1: The average interaction to next paint time of each type of renderer	13
Table 2: The average least contentful paint of the webpage	14
Table 3: The average response time of the server, including 'Content Download' time.....	14

Nomenclature

<i>JSON</i>	JavaScript Object Notation; a common data storage and transfer structure
<i>React</i>	A popular JavaScript framework which supports “components” and is used for creating web apps.
<i>Next.js</i>	A web development framework built on react which provides
<i>Leaflet.js</i>	A lightweight, open-source framework for displaying maps
<i>netCDF</i>	Network Common Data Form, a format designed for meteorological data
<i>GRIB</i>	General Regularly distributed Information in Binary form, a compressed format for meteorological data
<i>THREDDS</i>	Thematic Real-time Environmental Distributed Data Services
<i>OPeNDAP</i>	Open-source Project for a Network Data Access Protocol
<i>SVG</i>	Scalable Vector Graphic, a vector image format supported by HTML
<i>jsDAP</i>	A JavaScript Client for the processing of netCDF files which are sent over OPeNDAP

1. Introduction

Visualising weather data is hard. Trying to translate purely numerical representations of data to a decent understanding of the data is not something that the human brain was designed to do, and the existing tools to visualise outputs of weather models are slow, clunky, and support looking only at one file at a time. There is a gap in the current technology for a program which lets you visualise your own NetCDF or GRIB files in a way which is both easy to do and easy to understand.

Such a solution would require responsive feedback, real-time rendering of meteorological and oceanographic metrics, intuitive visuals and interactive layers, all of which should be provided through a simple yet powerful user experience. Such an experience can be provided through a TypeScript web project hosted on a containerised docker image, utilising *Next.js* with a *shadcn/ui* powered user interface (UI) and a *react-leaflet* based map renderer. This tech stack, as well as being more than adequate for meeting the requirements, has the added benefit of being free and open source, meaning that the project is able to be built upon and improved upon by open-source maintainers in the future.

This work will allow creators of weather forecasts as well as researchers and hobbyists to see the data that they have generated, as well as providing a base for the rendering of weather data not in netCDF or GRIB formats, which can be rendered by simply reshaping the data into the correct JSON accepted by the site.

The project requires the ability to render netCDF and GRIB files in real-time, using particle simulation for wind speed and direction visualisation, the ability to be deployed via a docker container, the ability to dynamically load data based on what data the user wants to view. If time permits, a backend server can be made to deliver the data to the frontend in a way that only provides the data that is required, cutting down on the transfer sizes between client and server by making use of caching. The project also requires a user interface which is intuitive to use, as well as responsive load times.

1.1. Literature Review

There are several similar real-world projects to the project being proposed, however almost all of them are closed source, making an in-depth review of the tech landscape challenging. Although we cannot inspect the implementation details of these projects, we are able to see the final product and make design choices and evaluate what works and what doesn't. As well as reviewing real world solutions, several studies have been conducted into a variety of techniques for rendering data that is consistent with the goals required for this project.

1.1.1. Data Transfer

One theme that was consistent between the literature is the issue of transferring data over the internet, with studies suggesting that three-dimensional visualisations can take up to 10MB per single time step (Treinish, 2002). Data transfer seems to be a key limitation of many of the solutions proposed in literature and has been solved using a variety of methods. One solution that has been leveraged is a local server to process the data directly from weather stations before serving it to the client (Xiaojuan Sun, 2012). Another method which has been used is the chunking of data on the server side before it is sent to the client, a technique which can be seen used by *windy.com* (Windy, 2014), which is revealed through reverse engineering (Figure 1). Another strategy that can be seen used in production is baking the data on the server and serving images to the client (Treinish, 2002) (Cowan, 2025)

(Pivotal Weather, 2025), which is a popular solution, but loses a lot of the interactability that comes with client side rendering.

```
1 {
2   "forecast": {
3     "-19.257/146.824": [
4       296,294,294, /*83 more values*/
5     ]
6   },
7   "reftime": 1759492800000,
8   "hours": [
9     3, 6, 9, /*105 more values*/
10  ]
11 }
12 }
```

Figure 1 An example of a minimised chunk used by windy.com

1.1.2. Visualisation Methods

Another widely covered topic is the technique used to render the data to the user. There are many varying techniques, each of which carries its own benefits and drawbacks. Many papers look at the viability of interactive three-dimensional rendering, which brings with it a great amount of interactability and informational density, as the user is able to see renders in all dimensions, as well as viewing the render from any angle. Such a system can be created using a grid-based system (Lu M, 2021), which allows for the level of detail to be dynamically adjusted based on the viewing area. Another way which three-dimensional rendering has been created is by making use of an existing three-dimensional platform (such as Google Earth) for rendering, and then displaying meteorological data based on altitude as layers above the map (Xiaojuan Sun, 2012). However, these designs are fundamentally limited by the rate of data transfer as discussed above, meaning that a strong connection is required for practical applications.

Another rendering technique that is used is a layered system rendered onto a two-dimensional projection of the globe, where different data can be rendered on different layers, as can be seen in various real-world solutions, such as *windy.com* (Windy, 2014) or Ventusky (as seen in Figure 2) (Ventusky, 2025), which sacrifices information density for a smaller data footprint, which is more important for systems which garner the number of users these sites do.

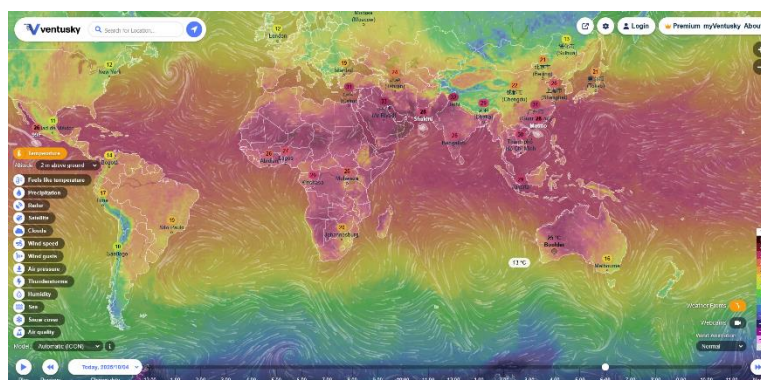


Figure 2: Ventusky's visual representation of weather from 04/10/2025

1.1.3. Technologies Used

There are a wide variety of programming languages used to generate visualisations of meteorological and oceanographic data. The most common language which is used is, as expected, JavaScript, which is what both Windy and Ventusky used for their rendering, with Windy using its own custom

modification of the leaflet.js library, the Map Forecast API. Another technique that can be used is to use JavaScript with WebGL to render the weather patterns (H Haase, 2000).

Outside of JavaScript, however, there are several novel approaches which are worth noting. One alternative method that was seen PHP server-based rendering, which allows for baking the image on the server instead of the client (as mentioned above)

1.2. Project Objectives

The primary objective of this project is to create a website that can dynamically render two-dimensional data using netCDF and GRIB files. The project aims to bridge the gap between the renderers which accept these files as inputs, and renderers which are web based, combining them in a way that makes it accessible for people who want to use local weather data without having to download programs specifically for viewing them, and who do not want to download the netCDF or GRIB files themselves.

This project serves to be incredibly useful to researchers, allowing them to properly visualise the data that meteorological stations are collecting, allowing them to more effectively understand the data that they are collecting. As well as researchers, this project will also progress the state of rendering GRIB and netCDF files in a way that is completely open source, allowing for future projects to build on the core that is created in this project.

2. Project Process

2.1. Constraints

This project, being a software project, does not have as many constraints as a traditional engineering project, with space, materials and (to some degree) cost bearing less relevance than a physical design project. Having said that, there are still significant design constraints, mainly in the form of time and software limitations. The project must be completed in approximately a year from assignment and will need to be able to be deployed to a docker container so that it can be hosted in production. As well as these constraints, the website must not take longer than five seconds to load, so that it performs to an acceptable standard.

As well as time and performance, many other key aspects of the site are constrained as well. The website will have access to the user's private data, such as their location, so it must also protect any personal information that it has access to, not recording anything without the user's explicit consent. The UI of the website must be easy to navigate, and users should be able to access the data that they want within five seconds of loading the page, so that they are able to quickly find the data that they want to view. The website must also be able to deal with erroneous or missing data gracefully, so that missing or corrupted data chunks from a weather station will not cause the entire website to crash. It is also important that the site is compatible with every modern desktop browser (excluding internet explorer).

2.2. Design Criteria

The criteria that this project will be assessed against have been compiled from meetings with the supervisor, as well as from the project description and the literature review. One of the key requirements is the ability for the website to be able to process both GRIB files and netCDF files from a remote server and extract the relevant data from each file type to render to the user on a two-dimensional map. It is also important that the extracted data can be filtered such that the data that is rendered can be selected by the user, to ensure that the user is not overloaded with information. A specific request from the client was for the render to feature a particle simulation for wind speed and direction, which will provide a visually intuitive representation of the wind-based data. Another requirement which was stated was for the rendering of the data to be real-time, meaning that you

don't need to wait excessive times to see the data that you requested, which is important for the usability of the website.

2.3. Software Stack

The software stack for this project will be written almost exclusively in TypeScript, making strong use of *React*, and *Next.JS*, using *react-leaflet* for map rendering and *shadcn/ui* for miscellaneous UI elements.

2.3.1. TypeScript

One of the projects requirements was that it must be web-based. Due to the way that the infrastructure for the web is designed, this decision already severely limits our options for a language. Outside of web-assembly based solutions, the only way to run code is on the net is through JavaScript. However, JavaScript, being a dynamically typed language, is prone to type errors and other problems which can be avoided by using TypeScript to ensure type safety. In response to this issue, in 2012, Microsoft created TypeScript, a superset of the JavaScript syntax which supports types. TypeScript works by taking a specific JavaScript version as a compile target and then compiling the TypeScript source code down to JavaScript. As a typed language, TypeScript allows for more compile-time error checking, which means that debugging will be much easier and will help to make the website much better at error handling.

2.3.2. React & Next.JS

In 2013, Facebook (now Meta) created React.js, which is a declarative JavaScript framework which allows for web design to be compartmentalised into “components”, which are smaller, reusable packages of JavaScript or TypeScript code which represent UI elements. React also allows for the abstraction of “state”, which allows for easier management of dynamic updating, taking the onus off the programmer to manage state updates. React is free and open source and has a large ecosystem of libraries built around it. This has been one of the key factors in its selection for this project, as well as its ability to increase development pace by reducing work spent on each page. To serve React pages to a web browser, a router is required. Next.JS is a router developed by Vercel, which was released in 2016, and has native support for React. Next.JS has support for server-side rendering, and makes for incredibly easy hosting via Vercel, as well as self-hosting. On top of this, it allows for hot-swapping code during development, which increases developer momentum.

2.3.3. Leaflet & react-leaflet

Leaflet.js (and its React port react-leaflet) is an open-source map rendering library with support for CSS in maps. It is used by many of the products seen in the literature review section, such as *windy.com* and *ventusky.com*. Leaflet is one of the only free and open-source map libraries, with alternatives such as the google maps API all being closed source or lacking in ability. Alternatives to leaflet which were considered for this project were projects that were built on leaflet such as Weacast or Kano, but ultimately working directly with leaflet allowed for use of react instead of working in a pure JavaScript based environment.

Another benefit of using Leaflet and react-leaflet is that Leaflet has a wide variety of plugins which can easily be ported to react-leaflet, such as leaflet-velocity and leaflet-boundary-renderer, which allow for using the work of others to improve both the user experience and development experience of the site.

2.3.4. Shadcn/ui

Shadcn is a relatively young library, being released in 2023. Since its release, it has taken over as the leading component library for React. It features sleek and performant general-purpose components,

providing a strong suite of tools for UI design. Its inclusion in this project is to allow for a heavier focus on the core aspect of the project, which is a map-based render, instead of the supporting UI.

Shadcn also has a graphing library which is based on recharts, a popular react library for graphing. This will allow for the display of data from specific points over time and will allow for a more natural way to interact with the data over time. On top of this, it will provide as a way to glean the underlying values of the data, as opposed to the general trend, which the other visualisation methods lend themselves to more. The benefit of having the graphing done through the UI library instead of doing it directly through recharts is that the theme of the graphs is consistent with the rest of the website. This allows for less time to be spent on styling and more time to be spent on improving the quality of the weather visualisation.

2.4. Design Evaluation

There are many challenges which are presented when trying to evaluate a data visualisation project, most of which stem from the inherent subjectivity of deciding the efficacy of a given representation. Bearing this in mind, there are plenty of functional and non-functional metrics which can be used to judge the final design of the project.

One of the key metrics that can be used is the load time of the site. Measuring the time from webpage request to final data visualisation can demonstrate the overall performance of the backend, which serves the THREDDs data to the frontend, as well as the data-processing of the frontend. This metric can be broken down into two sub-categories; the time it takes for the frontend to receive the data from its request, and the time the frontend takes to render the data once it has been received. These two categories can be measured by measuring the timing from the first request to the rendering of the data, then using the built-in developer tools in the browser to obtain the request round trip time, then subtracting this number from the overall time to calculate the render processing time for the frontend.

The performance of the frontend can also be measured using Google Chrome's developer tools (Google, 2024), which provide an insight into the performance of the visualisation once the data is loaded. Google Chrome offers three metrics, namely the largest contentful paint (LCP), cumulative layout shift (CLS), and interaction to next paint (INP). The CLS measures the amount that the layout of the site shifts by, which is not a relevant metric to our site, as its layout is entirely static. The LCP and INP are much more relevant, with the INP measuring the amount of time the site takes to update after being interacted with, and the LCP measuring how long the site takes to render the most significant piece of content.

2.5. Health & Safety Considerations

Due to the nature of the project, there are not very many health and safety considerations to keep in mind. Regardless of this, a risk assessment has been undertaken for this project, and the few risks that do exist have been judged against the risk assessment matrix provided by the Western Australian Government (Western Australian Government, 2025).

2.5.1. Misinformation Leading to Action/Inaction

Description

If the software is to produce an erroneous weather forecast, it may cause distress to individuals who see potential catastrophic weather events which do not exist. Alternatively, it could fail to report a serious weather event and lull people into a false sense of security.

Likelihood

For this to occur, either an erroneous reading would need to be hallucinated by the website itself, or a bad reading would need to be produced by the forecasting body. On top of this, the erroneous reading would need to present as a realistic weather phenomenon or would need to coincide with a

real catastrophic weather event. Whilst not impossible, this seems quite unlikely and hence receives a likelihood rating of unlikely (L2).

Consequences

The consequence of a misrepresentation of the weather system could have a real impact on many aspects of someone's life. A misreported weather pattern could lead people to leave their house in a situation that is unsafe, or cause distress to a person who believes that a catastrophic weather event is coming when it is not.

Management Strategy

Multiple management strategies can be implemented to avoid issues with misrepresented data. The first solution is an engineering solution, where the program is thoroughly tested to ensure that it is capable of accurately reporting weather systems. On top of this, an administrative mitigation strategy can be employed by simply placing a warning at the top of the page reminding the user that they should always cross reference the weather conditions seen, and in an emergency to visit the correct channels.

2.5.2. User's Sensitive Data is Leaked

Description

Because the website will be dealing with user's personal data, such as their location, it is possible that a data breach occurs, which reveals in user's private information becoming public.

Likelihood

Without a consideration for privacy, and adequate planning around the handling of sensitive data, an issue like this would not be exceedingly unlikely to occur, so it will receive an unlikely (L2) for its likelihood rating.

Consequences

While the risk matrix in the appendix is designed around physical risks, the consequence of leaking user data is quite severe, involving multiple regulatory bodies preparing to impose financial penalties, as well as the effect on people's livelihoods when they are effectively Doxed¹.

Management Strategy

The risk of dealing with personal data can be almost entirely eliminated by keeping the user's personal data completely on the frontend, which means that the website never saves their location and thus it cannot be traced. As well as this, only using the user's location if it is absolutely necessary will also serve to eliminate the risk that the user's location is leaked.

¹ "Doxing" is the act of releasing someone's address to the internet so that people can do bad things to them (threaten them or call law enforcement with bomb threats (colloquially "SWATting"))

3. Results and Discussion

3.1. Final Design

The source code for the final project can be found on GitHub (Healy, 2026), and is completely open source under the MIT licence. The final design of the weather visualiser consists of a NextJS frontend, which has only one page which renders the data on the browser, and a NextJS Backend, which is responsible for fetching the data from the THREDDS server, caching it and formatting it for the frontend.

The backend of consists of one simple endpoint, called `/api/getDataFromThredds`, which uses the date and requested source, fetches the data from the server, processes it to be in a usable form for the frontend, caches it and then sends it to the frontend.

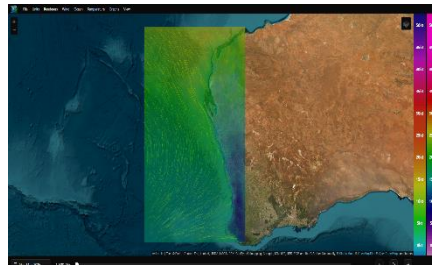


Figure 3: The final renderer, representing the windspeed and the surface temperature

The frontend consists of a menu bar, a timeline, and a map (**Figure 3**). The menu bar consists of all the settings, which allows for selection of the data source and the required visualisations. The timeline is simply a calendar selector and a slider which allow for the selection of a time and date for the data to be visualised. The map consists of a world map which can either be set to a satellite map or a street map, and contains the visualisations for the wind, current and temperature data. The wind data can be rendered using a particle renderer, a wind barb display and a colour map which shows speed as colour. The temperature can be rendered using similarly to the wind speed colour map but using a scale which is more adept at handling temperature. Similarly to the wind data, the current can be displayed using either the particle renderer or using arrows.

3.1.1. Particle Renderer

The main way that both the current and wind data are represented is through the particle renderer. The particle renderer makes use of a custom written wrapper around the leaflet-velocity plugin, which it integrates via the react-leaflet core library, which allows for raw leaflet layers to leverage the features of react and react-leaflet such as react state, and allows for them to interface with the constructs set out in react-leaflet. The leaflet-velocity map layer requires a JSON representation of a GRIB header (as produced by wecast-grib2json (Weacast, 2026)), which it receives from the server.

The particles shown themselves moving in accordance with the vector field that represents the wind magnitude and direction. The colours of the particles are selected based on the magnitude of the particle, with the wind simulation using the same colour map as the wind colour overlay, and the current using the built-in velocity colour map.

The colours of the wind also align with the colour scale which is displayed if the ‘Show Color Scale’ option in the ‘Wind’ menu. The particle renderer shows the movement of the wind at a given time step, using time to show the direction of the wind. This means that it is difficult for the renderer to show the data over time, as it already uses the time dimension to show the direction. This leads to the data being hard to parse when it is played back using the timeline feature.

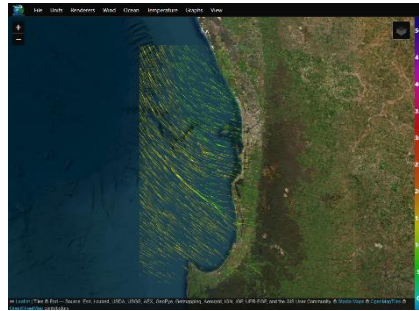


Figure 4: The particle renderer showing wind data on April 1st of 2026

3.1.2. Arrows and Wind barbs

One way that the program visualises the data is using SVG based pictograms. The program uses wind barbs and current arrows to represent the wind and current values respectively, as these values are the only values that are vectors, and are hence the only values which require the representation of both magnitude and vectors. Unlike the particle renderer, the SVG based renderers don’t use time as an axis for data display, making them much more useful when viewing the data over time. For performance reasons, as well as for the ability to correctly order the layers, all the SVG renderers (excluding the water-based renderers) are inside of a single react-leaflet SVG Layer, which saves on processing overhead for multiple layers, and allows for precise control of which layers render on top of other layers. The arrows and wind barbs are rendered above the layers colour map layers where possible, within the exceptions set out in the section on oceanographic data rendering.

Wind barbs (**Figure 5**) are a standard way to display anemological data (NOAA, 2026), that make use of a stem which consists of several tails which are used to indicate the windspeed. If the wind speed is less than five knots, a circle is displayed centred on the point the barb represents. If it is larger than five knots, a stem pointing in the direction that the wind is coming from is drawn, with tails representing 10 knots, and a half tail being used to indicate five knots. If the wind speed is greater than 50 knots, a triangle is used on the stem instead of five tails. The data for the wind barbs is calculated the same way that the colour map is calculated, splitting the visible data into squares, averaging the data inside each square and using that to calculate the wind direction and speed.



Figure 5: Wind barbs for 0, 5, 10, 15, 20, 50, 55, 60 and 100 knots

The wind barbs are also drawn using SVG, and similar to the colour map, because their resolution is tied to the visible data, they will automatically resize themselves such that their size on the screen stays the same proportionate to the data that is in view. The wind barbs are also useful for comparing the data being represented with other sources of anemological data, as being standardised means that other sources, such as XY-Grib (OpenGribs, 2026), an open-source program designed to view grib files, which was used to test the validity of the data processing and representation. One shortcoming of the wind barbs is that it can be difficult to parse the magnitude of the wind at a glance, as a magnitude of 15 to 35 is only represented by two tails – meaning that when the data is shown for a short, like when data over time is shown, it can be hard to discern the trends of windspeed.

The arrows used to represent the current are rendered in much the same way. The arrows are used to show the way that the oceanic currents are moving, with the size of the arrow being used to demonstrate the magnitude of the current. These arrows are rendered in either light blue or dark blue, based on the colour of the background for visibility. The current data given for each region is smaller than the wind and temperature data, which means that the current data averager needs to be able to deal with empty values, which are represented as 'NaN' values, a floating point number which represents something that is not a number. To deal with this, when the data is averaged for the square the current arrow is contained in, the amount of empty values is stored, and the data is discarded if a certain threshold is met.

3.1.3. Heatmaps

Heatmaps are the third way that the data can be displayed on the map. The three datapoints that are heat mapped are the surface temperature, the ocean temperature and the surface windspeed. The temperature naturally lends itself to the being displayed by heatmap, but the decision to render the windspeed is more novel. The windspeed being rendered as a heatmap is a direct product of the shortcomings of the particle rendering and the wind barbs. As mentioned above, there is a fundamental limitation for using the wind particles over time, and the magnitude can be hard to glean from the wind barbs when they are changing at pace. The windspeed heatmap aims to fix this by providing a representation of magnitude in a way that is easy to understand quickly.

The heat maps lend themselves to displaying the data over time, similar to the SVG based pictograms, as they do not make use of time to display any data. The heat maps work by using an SVG layer, provided by react-leaflet, which spans the width of the data frame that is visible, and is then divided into equal squares based on the 'Resolution' setting which is provided by the wind menu. These squares then average all temperature values which land within their bounds and show this averaged value. The number of squares is tied to the amount of the data visible on the screen, with the 'Resolution' setting determining the maximum number of squares on the larger edge of the data boundary. This means that as the user zooms in, each square becomes more accurate, as each square will be smaller relative to the total area of the data.

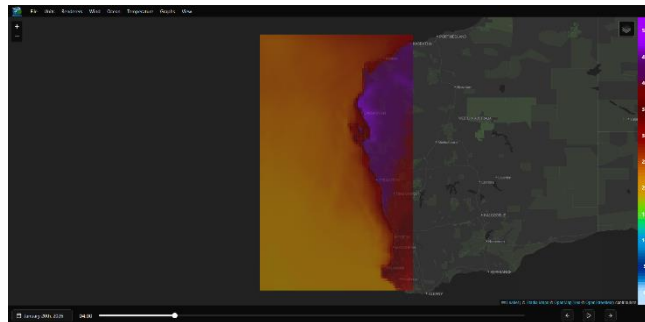


Figure 6: Temperature data being displayed for the 20th of January (Hottest day of 2026 at time of writing)

There are two colour scales which the program makes use of: one scale for the wind speed, and one scale for the temperature. Both colour scales are not linear but are instead hand-crafted to best show the variance in values. The temperature colour scale is generated using a set of key points for hue, luminance and chroma individually. These key points were selected based on the work presented by the Environmental Systems Research Institute (ESRI), which gives directions on how to create a strong colour gradient using variance in lightness as well as hue (ESRI, 2022). The colour gradient which is generated makes use of the OKLCH colour space, which is a colour space similar to HSV, but is perceptually linear (Wikipedia, 2026), meaning that the changing the hue value without changing the luminance value does not affect the perceived brightness to the human eye. In contrast to this, HSV is not perceptually linear, meaning that, the brightness of colours in the blue to green

and green to yellow sections of the colour spectrum appeared different to that in the yellow to red and red to magenta areas, making controlling the lightness by hand more difficult.

On top of this issue, the temperature scale also suffered from a lack of total colours. As temperatures can realistically range from -5 to 45 in Western Australia, the scale needed to be able to handle these extremes without wrapping back on itself, whilst still being able to show the difference between common values with enough resolution that they can be meaningfully interpreted. To solve this problem, the colour scale also makes use of the saturation value, to fade low temperatures from dark blue into white, and to limit the hue of high temperatures so that they never swing past purple, but instead use the luminance and chroma to display the difference in values at the extremes (as can be seen in **Figure 6**).

3.1.4. Coastline Interaction

One of the biggest issues that wasn't anticipated was the issue of oceanographic products not accounting for the boundary between land and ocean. Because of the average based rendering of both the current arrows and the ocean temperature heatmap, data points which were on the edge of the coastline of the mainland and it's surrounding islands would sometimes render over land, where they should not be viewed as they are nonsensical. Compounding this, the dataset representing the greater Western Australian Coast does not appear to consider Rottnest Island a land mass, as it has values populated for current and ocean temperature. Furthermore, the technique to translate the curvilinear grid to a pure linear grid set out in section 3.1.1 leads to even more values where they do not belong. This issue was one of the hardest to overcome, and the solution which was settled on was not a satisfying one, leaving plenty of room for future improvement.

The first attempt which was taken to try to rectify the issue was to take a map of the Australian coastline, which the Australian Government provides free of charge (Australian Government, 2026), and use that to cut datapoints which are inside the coastline and then only render those values which don't land inside the shape. However, this came with many drawbacks. Firstly, the data provided by the government is in shapefile form, which does not have wide support in JavaScript, which has a much stronger support for GeoJSON (IETF, 2016), which is a specification for a JSON format specifically for geological representation.

Converting to GeoJSON is not an impossible hurdle to overcome, but the next issue was the size of the file. When converted to GeoJSON, the coastline was upwards of 40MB, which was causing performance issues when it was being loaded. To mitigate this, the file is now pre-processed into the area which overlaps with the largest data window that is on the THREDDS server, and all islands which are too small are thrown out to reduce file size.

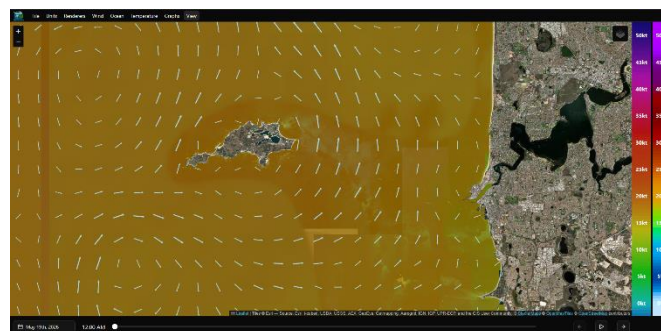


Figure 7: The ocean temperature and current arrows being drawn behind Rottnest

The final issue with this approach, which proved insurmountable, was the performance of checking if a given point was inside the land. The complex geometry of the West Australian coast meant that checking each point was in the hundreds of milliseconds, which compounded quickly considering that the smallest data window contained approximately 14 thousand points. The performance was

such a bottleneck that even after creating a bounding box around coastal areas, first checking points against the simplified bounding box before checking them against the complex geometry proved too expensive to meet the real-time rendering requirement imposed in section 2.2.

The final solution to this issue involved using a library called leaflet-boundary-canvas. The library is capable of taking a GeoJSON feature and using it to mask a leaflet tile layer, which is the base layer of the map. This allowed for the rendering of the land outlined in the GeoJSON file on top of the data, allowing for the data which should not be displayed to be obfuscated by the second rendering. Using react-leaflet panes, the rendering order of the layers can be shifted such that only the oceanographic products are rendered below the boundary layer, which means that the arrows from the current layer are not able to be rendered on top of the temperature and windspeed heatmaps. This solution is not optimal, however, as it means that data which is outside of the area covered by the GeoJSON will not correctly render, and the GeoJSON file would need to be manually update in the event of a major coastline change of Western Australia.

3.1.1. Point Based Data Representations

The site also comes equipped with tools for viewing the data at a specific location, supporting both a data display based on react-leaflet's tooltip as well as a graphing-based representation built on recharts. These serve to supplement the existing renders, allowing for specific numbers to be extracted from the visualiser.

The mouse-over display leverages the leaflet tooltip to render all relevant data in the units selected by the user at the mouse. The way that these datapoints are calculated is different to the other renderers, and depends on the if the user has the 'Interpolate Mouse Data' setting enabled. If the user is using the interpolation setting, the point is calculated using a bilinear interpolation between the four closest grid points for each frame that the mouse location falls within. This serves to give smooth readings grid points, meaning that datasets with a large separation between points would remain continuous. When the interpolation is disabled, the data mouse-over simply renders the closest data point. This is more useful for denser datasets, as here the difference between points is small enough that the interpolation only serves to make readings less accurate.

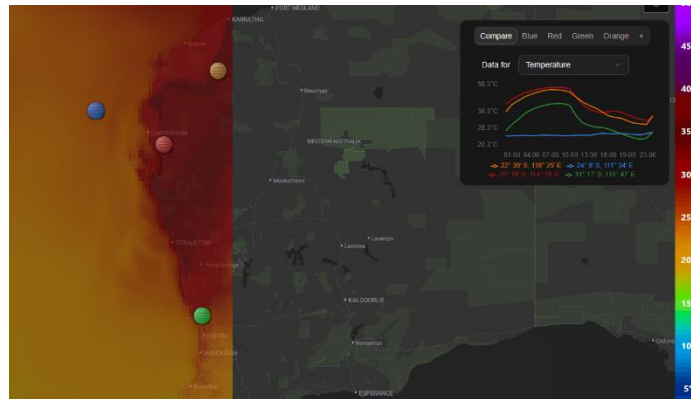


Figure 8: A closeup of the point grapher, comparing the temperature at different points

On top of the mouse over renderer, the website also has a point grapher (**Figure 8**), which is able to show the trends over time for all datasets, as well as comparing these values for up to five different points. By default, the graph tries to fit all the data, automatically adjusting its axes to show the best data definition. However, it can be forced to have zero as the minimum for the y axis, which helps to show the value of the data being displayed, as opposed to the shape of the data.

3.1.2. Backend Data Processing

The way that the webpage receives the data is through an intermediate backend which is hosted on the same server as the frontend. This design decision is to allow for a caching system, as well as to

allow for avoiding cross-origin resource sharing issues (CORS) from the backend. When the user requests the data for a specific day and region, the backend first checks if it has previously fetched and processed the relevant data. The data is cached by saving the final data into the 'cachedResponses' folder as a JSON file. If the data is not in the cache, it then goes out to fetch the relevant data from the THREDDS server using jsDAP (stcorp, 2006), a web requesting wrapper that is designed for accessing netCDF files through OPeNDAP. The data is requested based on a hardcoded list of parameters which correspond to the specific parameters the visualisation requires. The data then gets reshaped into an array of GRIB frames, which the particle renderer requires. If the grid used by the netCDF data is linear, this conversion is simple, but if it is curvilinear, there are extra steps required to coerce the data into a linear grid.

The process of turning the curvilinear grid into a traditional grid was another point of friction for the project, as many of the early solutions suffered from being highly performance intensive, which lead to unreasonable wait times from the client. The first issue was that the amount of data received for the larger area's oceanographic metrics was enough to almost immediately crash the backend during processing. Because of the sheer volume of data that requesting every point requires (somewhere in the order of 200 megabytes), the server requests every third latitude point and every fourth longitude point for the largest data set (the ocean data for the whole WA coast), with an option for the user to half the amount of data received in each axis called 'Quick Load'. Once this hurdle was overcome, there was the issue of aligning the grid. One method that was attempted was calculating the closest data value to each output latitude and longitude and using that value, but that was too slow to be practical. The next method which was implemented was the same process but executed the other way around; snapping each point in the input array to its closest output point. Whilst much more performant, this left rows and columns of unpopulated data, where the points around them had all snapped away from them, leaving them valueless. The final solution involved using the same snapping to the closest grid point, and filling these holes with the averages of the points which are around it, allowing for a performant solution that, whilst not perfectly accurate, leaves no empty values and is close enough that a visual representation of the data is essentially identical to that of the complete data.

Once the realignment of the grid is done, the grid is combined with a generated GRIB header which includes the fields that the renderers require, all of the values are rounded to four decimal places, then the file is saved to the aforementioned 'cachedResponses' folder as a JSON file. When the file is added to the cache, it checks to see the number of files in the cache, and if it is greater than 40, it deletes the oldest file. As the average file size is approximately 20 megabytes, the cache can get up to approximately one gigabyte. The cache is also kept in a docker volume, meaning that it is persistent between sessions. The backend also automatically polls the next 6 days' worth of data for both the perth and greater perth regions every 12 hours, to get files which are most likely to be required to be preloaded to save on loading times.

3.1.3. *UI and General Settings*

The user interface of the website is fairly minimal, as most of the attention of the user is going to be focused on the map. It consists of a menu bar which contains:

- A 'File' menu, where you can select the data source for the visualiser;
- A 'Units' menu, where you can select the units for the data display;
- A 'Renderers' menu, where you can select the current data visualisation methods;
- 'Wind', 'Ocean' and 'Temperature' menus, where you can configure the settings for each category of visualisation;
- A 'Graphs' menu, where you can enable and configure the point grapher; and
- A 'View' menu, where you can configure various viewing related options.

These menus are all displayed as a top bar with drop downs, and all responsively display the current value of the setting that they control. These settings are stored in the browser's `localStorage`, which

allows for it to be persistent without requiring it to be stored on the server. The settings are then revealed to the program using a react context, which makes them accessible wherever they might be needed.

The ‘View’ menu contains the option to demarcate the boundary of the data, which simply draws a square around the edge of the largest data frame for the given data and data source. This was mainly used for testing, but there was no cost to keeping it as an option, so it was not removed. The view menu also contains the dark mode setting, which can be used to set the site to either light mode or dark mode, to help with the user experience for people using the site in low-light environments.

On top of these settings, there are the controls which come native with Leaflet, which are the layer selector and the zoom controls, which allow for toggling between the satellite view and the street view and changing the zoom on the map respectively. The selected map is also persistent between sessions.

3.2. Performance and Analysis

3.2.1. Rendering metrics

Measuring the largest contentful paint and the interaction to next paint involved two different testing techniques, as the LCP is determined by the initial load, whereas the INP is determined by use of the site. The values are compared against the values which google recommends for a good user experience (Google, 2024), providing an industry accepted reference frame – although it is important to note that these values are for normal webpages, and such are tuned for a less performance intensive task than weather visualisation.

To get values for the INP, all types of renderers are tested using the wind visualisation, as the wind visualisation represents every type of renderer utilised by the project. Individual renderers are tested by loading the page with all renderers disabled and all other settings left as default. Once the page is loaded, the renderer is enabled, the appropriate settings are set, and the simulation is tested by moving the map such that it is zoomed in on Rottneest, then back out, all whilst moving the map around to register inputs. The INP, which is the main metric that can change once the page has loaded, is recorded as the longest time between an input and the page re-rendering, meaning that the metric measures the moment of worst performance throughout the test. The test is repeated five times per renderer, and the full set of values as well as testing conditions can be seen in Appendix A: Raw Data for Performance Measurements. Most of the renderers perform better than the bounds set, with the exception of the particle renderer and the heatmap at max settings. Particle simulation is inherently expensive, and it is of no surprise that the particle simulation represents the most expensive part of the simulation. The heatmap being above the threshold for error is not surprising either, as the maximum resolution is set quite high for the heatmap renderers, and a warning about the performance implications of setting these values to their extremes is provided to the user.

Table 1: The average interaction to next paint time of each type of renderer

<i>Test Performed</i>	Average INP (ms)
<i>Acceptable benchmark (from web.dev)</i>	200
<i>Particle Renderer</i>	318.8
<i>Heatmap at default settings, eight times speed</i>	43.2
<i>SVG Drawing at default settings, eight times speed</i>	99.2
<i>Heatmap at maximum settings, 16 times speed</i>	57.6
<i>SVG Drawing at maximum settings, 16 times speed</i>	260.8

The LCP is recorded by simply loading the page in google chrome, recording the value (which is reported in the developer settings) then clearing all browsing data from the last hour before the next test to avoid any issues with caching. The time with caching is also recorded and follows the same process as the non-caching tests but without clearing the history in between. The average LCP which is recorded is well below the recommended LCP, which makes sense as the website rendered is quite simple; the performance demands come primarily from processing the data received, not from the rendering complexity of the site.

Table 2: The average least contentful paint of the webpage

<i>Test Performed</i>	<i>Average LCP (s)</i>
<i>Acceptable benchmark (from web.dev)</i>	2.5
<i>LCP without caching</i>	1.364
<i>LCP with caching</i>	0.364

Overall, the site exceeds the metrics set in most categories, with only the particle rendering causing significant performance issues for the client.

3.2.2. Load times

One of the main issues that this project has had to combat is the sheer size of the data leading to poor load times resulting from high processing times as well as high data transfer times. These load times can be measured using the developer console in chrome again and are recorded as the sum between the ‘Waiting for server response’ time and the ‘Content Download’ time. These times can be used to judge the speed of the server and the size of the payload respectively, as the waiting time is resultant from the server fetching from the cache. The times that we see are quite high, which is to be expected when dealing with large datasets. However, times this large are likely the result of an underlying structural problem, and an indication that a backend written in a more robust language might be a more appropriate solution than the one provided.

Table 3: The average response time of the server, including ‘Content Download’ time

<i>Test Performed</i>	<i>Average Time (s)</i>
<i>Time to load data</i>	3.196

4. Conclusions and Future Work

This project represents a cursory exploration of rendering weather data from a THREDDDS server. The project achieves all key goals, being able to render the data in a manner that is both real time and informative, with support for a particle visualisation as well as graphing the data at a specific point. The site also contains a backend server, which is capable of caching the data received from the THREDDDS server, to cut down on the traffic that the THREDDDS server receives as well as to provide more responsive loading times to the frontend. It is performant within acceptable bounds, with most of the rendering methods landing within the acceptable parameters of a website.

The project will meaningfully contribute to any research requiring meteorologic or oceanographic analysis, providing the ability to render forecasting data from a THREDDDS server in a way that is much more ergonomic than previous approaches, cutting out the need for the user to access the THREDDDS backend directly. It is also open source, meaning that the project is easily extensible to anyone who needs to render THREDDDS data, or any other data which can be shaped into the format required by the renderer

There are many directions in which this project could be extended. One of the main improvements which could be made would be to extract the backend of the project into a separate server, which could then be written in a compiled language such as rust or C#, which would provide a massive

speed up compared an interpreted language like JavaScript. This speed up could then be used to take advantage of a more accurate system for aligning the curvilinear grid seen in the ocean data, as the extra speed would allow for a slower but more precise approach. If significant enough, the extra speed may be able to be leveraged to properly filter coastline data on the server side, removing the need for the extra layers that are rendered on the frontend.

Another direction which could be taken would be to implement rendering for other values, such as salinity and rainfall, both of which are available from the dataset used. These metrics could be easily rendered using the existing heatmaps, and although they are beyond the scope of this project, these visualisations would help to further optimise the experience, saving the end user from going back to the THREDDS server unnecessarily.

5. References

- Australian Government. (2026, 05 16). *Australian Coastline 50K 2024 (NESP MaC 3.17, AIMS)* . Retrieved from Data.gov.au: <https://www.data.gov.au/data/dataset/australian-coastline-50k-2024-nesp-mac-3-17-aims>
- Cowan, L. (2025). *Numerical Model Prediction*. Retrieved from Tropical Tidbits: www.tropicaltidbits.com/analysis/models/
- ESRI. (2022, 04 26). *Creating a meaningful temperature palette* . Retrieved from ArcGIS Blog: <https://www.esri.com/arcgis-blog/products/arcgis-pro/mapping/a-meaningful-temperature-palette>
- Google. (2024, 09 17). *Monitor your local and real-user Core Web Vitals performance in DevTools*. Retrieved from Chrome for developers: <https://developer.chrome.com/blog/devtools-realtime-cwv>
- H Haase, M. B. (2000). Meteorology meets computer graphics — a look at a wide range of weather visualisations for diverse audiences. *Computers & Graphics*,, 391-397.
- Healy, J. (2026, 05 14). *WeatherForecastVisualiser*. Retrieved from Github: <https://github.com/jh1236/WeatherForecastVisualiser>
- IETF. (2016, 08). *The GeoJSON Format*. Retrieved from Datatracker: <https://datatracker.ietf.org/doc/html/rfc7946>
- Lu M, W. X. (2021). Web-based real-time visualization of large-scale weather radar data using 3D tiles. *Transactions in GIS*, 25-43.
- NOAA. (2026, 05 14). *Information about wind barbs*. Retrieved from National Weather Service: <https://www.weather.gov/hfo/windbarbinfo>
- OpenGribs. (2026, 05 15). *XYGrib*. Retrieved from OpenGribs: <https://opengribs.org/en/xygrib>
- Pivotal Weather. (2025). *WPC Forecasts - Pivotal Weather*. Retrieved from Pivotal Weather: https://www.pivotalweather.com/maps.php?ds=wp&p=wp_excessive_rainfall_day1&r=conus
- stcorp. (2006, 07 28). *jsDAP*. Retrieved from Github: <https://github.com/stcorp/jsdap>
- Treinish, L. A. (2002). "Case study on the adaptation of interactive visualization applications to Web-based production for operational mesoscale weather models,. *IEEE Visualization*,, 549-552.
- Ventusky. (2025). *Ventusky - Weather Forecast, Radar and Wind*. Retrieved from Ventusky: <https://www.ventusky.com/>
- Vercel. (2025, 10 1). *Vercel*. Retrieved from Vercel Pricing: <https://vercel.com/pricing>
- Weacast. (2026, 05 14). *@weacast/grib2json*. Retrieved from GitHub: <https://github.com/weacast/weacast/tree/master/packages/grib2json>
- Western Australian Government. (2025). *Template 2.16: Risk Assessment*. Retrieved from Western Australian Government: <https://www.wa.gov.au/government/publications/template-216-risk-assessment>
- Wikipedia. (2026, 05 15). *HSL and HSV*. Retrieved from Wikipedia: https://en.wikipedia.org/wiki/HSL_and_HSV#Disadvantages

Windy. (2014). *Windy.com*. Retrieved from Windy: <https://www.windy.com/>

Xiaojuan Sun, S. S. (2012). Development of a Web-based visualization platform for climate research using Google Earth,. *Computers & Geosciences*,, 47, 160-168.

6. Appendices

Appendix A: Raw Data for Performance Measurements

1. LCP Data for page when Cached vs First load

Test	1st LCP (s)	2nd LCP (s)	3rd LCP (s)	4th LCP (s)	5th LCP (s)
<i>First Load</i>	1.41	1.36	1.38	1.38	1.29
<i>Cached</i>	0.36	0.37	0.36	0.36	0.36

2. Data for Individual Renderer at Default Settings

Renderer	1st INP (ms)	2nd INP (ms)	3rd INP (ms)	4th INP (ms)	5th INP (ms)
<i>Particle</i>	304	248	370	424	248
<i>SVG Drawing</i>	48	48	40	40	40
<i>Heatmap</i>	88	88	104	112	104

3. Data for Individual Renderer at Maximum Settings

Renderer	1st INP (ms)	2nd INP (ms)	3rd INP (ms)	4th INP (ms)	5th INP (ms)
<i>Particle</i>	-	-	-	-	-
<i>SVG Drawing</i>	64	56	56	56	56
<i>Heatmap</i>	224	304	272	256	248

(Values Omitted from particle renderer as performance settings are not present)

4. Data for All renderers Enabled at Maximum Settings

Renderer	1st INP (ms)	2nd INP (ms)	3rd INP (ms)	4th INP (ms)	5th INP (ms)
<i>All</i>	1080	864	1016	928	952

5. Data for Combined Server-Client Load Times

1st load (s)	2nd load (s)	3rd load (s)	4th load (s)	5th load (s)
3.13	3.27	3.23	3.20	3.15

6. Information for Test Environment.

System Information is taken from msinfo32.exe, google chrome information is from the help page in settings.

Specification	Value
System Information	
<i>OS Name</i>	Microsoft Windows 11 Pro
<i>Version</i>	10.0.22000 Build 22000
<i>System Manufacturer</i>	Micro-Star International Co., Ltd.
<i>Processor</i>	AMD Ryzen 5 5600X 6-Core Processor, 3701 Mhz, 6 Core(s), 12 Logical Processor(s)
<i>Installed Physical Memory</i>	16.0 GB
<i>Total Physical Memory</i>	15.9 GB

Total Virtual Memory

21.2 GB

Page File Space

5.25 GB

Chrome Information

Google Chrome Version

148.0.7778.179 (Official Build) (64-bit)